

第6章 计算机的运算方法

6.1 无符号数和有符号数

6.2 数的定点表示和浮点表示

6.3 定点运算

6.4 浮点四则运算

6.5 算术逻辑单元



三、乘法运算

- 乘法运算的实现：硬件乘法器、软件编程
- 为何学习乘法运算：乘法器设计、乘法编程

三、乘法运算

1. 分析笔算乘法

$$A = -0.1101 \quad B = 0.1011$$

$$A \times B = -0.10001111 \quad \text{乘积的符号心算求得}$$

$$\begin{array}{r} 0.1101 \\ \times 0.1011 \\ \hline 1101 \\ 1101 \\ 0000 \\ 1101 \\ \hline 0.10001111 \end{array}$$

✓ 符号位单独处理

✓ 乘数的某一位决定是否加被乘数（包含多次被乘数左移）

✓ 将4个位积一起相加；难以实现

✓ 乘积的位数扩大一倍；增加硬件成本及运算时间



2. 笔算乘法改进

$$A \cdot B = A \cdot 0.1011$$

两数相乘转
化为迭代的
加法和移位
计算

$$= 0.1A + 0.00A + 0.001A + 0.0001A$$

$$= 0.1A + 0.00A + 0.001(A + 0.1A)$$

$$= 0.1A + 0.01[0 \cdot A + 0.1(A + 0.1A)]$$

右移一位

$$= 0.1\{A + 0.1[0 \cdot A + 0.1(A + 0.1A)]\}$$

$$= 2^{-1}\{A + 2^{-1}[0 \cdot A + 2^{-1}(A + 2^{-1}(A + 0))]\}$$

第一步 0 + 被乘数

第二步 右移一位，得新的部分积

第三步 部分积 + 被乘数

⋮

第八步 右移一位，得结果

B数值位有4位，
加4次，移位4次

①

②

③

⑧



3. 改进后的笔算乘法过程（竖式）

部分积	乘数	说明
$\begin{array}{r} 0.0000 \\ + 0.1101 \\ \hline \end{array}$	$101\underline{1}$	初态，部分积 = 0 乘数为 1，加被乘数
$\begin{array}{r} 0.1101 \\ 0.0110 \\ + 0.1101 \\ \hline \end{array}$	$110\underline{1}$	→ 1，形成新的部分积 乘数为 1，加被乘数
$\begin{array}{r} 1.0011 \\ 0.1001 \\ + 0.0000 \\ \hline \end{array}$	$11\underline{10}$	→ 1，形成新的部分积 乘数为 0，加 0
$\begin{array}{r} 0.1001 \\ 0.0100 \\ + 0.1101 \\ \hline \end{array}$	$111\underline{1}$	→ 1，形成新的部分积 乘数为 1，加被乘数
$\begin{array}{r} 1.0001 \\ 0.1000 \\ \hline \end{array}$	1111	→ 1，得结果



小结

- 乘法 运算可用迭代的 加和移位实现
 $n = 4$ ，加 4 次，移 4 次，每次迭代如下：
 - 由乘数的末位决定被乘数是否与原部分积相加，
然后 $\rightarrow 1$ 位形成新的部分积，同时 乘数 $\rightarrow 1$ 位
(末位移丢)，空出高位存放部分积的低位
 - 被乘数只与部分积的高位相加
- 所需硬件 3 个寄存器，具有移位功能
 1 个全加器



4. 原码乘法

(1) 原码一位乘运算规则

以小数为例

$$\text{设}[x]_{\text{原}} = x_0.x_1x_2 \cdots x_n$$

$$[y]_{\text{原}} = y_0.y_1y_2 \cdots y_n$$

$$\begin{aligned}[x \cdot y]_{\text{原}} &= (x_0 \oplus y_0).(0.x_1x_2 \cdots x_n)(0.y_1y_2 \cdots y_n) \\ &= (x_0 \oplus y_0).x^*y^*\end{aligned}$$

式中 $x^* = 0.x_1x_2 \cdots x_n$ 为 x 的绝对值

$y^* = 0.y_1y_2 \cdots y_n$ 为 y 的绝对值

乘积的符号位单独处理 $x_0 \oplus y_0$

数值部分为绝对值相乘 $x^* \cdot y^*$



(2) 原码一位乘递推公式

$$x^* \cdot y^* = x^*(0.y_1y_2 \dots y_n)$$

$$= x^*(y_12^{-1} + y_22^{-2} + \dots + y_n2^{-n})$$

$$= 2^{-1}(y_1x^* + 2^{-1}(y_2x^* + \dots 2^{-1}(y_nx^* + 0) \dots))$$

The diagram illustrates the recursive formula with nested brackets. A large green bracket under the entire expression is labeled z_n . Inside this bracket, there is an ellipsis $\dots$ and a smaller green bracket labeled z_1 . Above the z_1 bracket, there is another smaller green bracket labeled z_0 that encloses the term $2^{-1}(y_nx^* + 0)$.

$$z_0 = 0$$

$$z_1 = 2^{-1}(y_nx^* + z_0)$$

$$z_2 = 2^{-1}(y_{n-1}x^* + z_1)$$

$\vdots$

$$z_n = 2^{-1}(y_1x^* + z_{n-1})$$



例6.21 已知 $x = -0.1110$ $y = 0.1101$ 求 $[x \cdot y]_{\text{原}}$

解: 数值部分的运算

部分积		乘数	说明
0.0000		<u>1101</u>	部分积 初态 $z_0 = 0$
+ 0.1110			+ x^*
逻辑右移	0.1110	<u>0110</u>	$\rightarrow 1$, 得 z_1
	+ 0.0000		
逻辑右移	0.0111	0	$\rightarrow 1$, 得 z_2
	+ 0.0000		
逻辑右移	0.0011	<u>1011</u>	$\rightarrow 1$, 得 z_3
	+ 0.1110		
逻辑右移	1.0001	10	$\rightarrow 1$, 得 z_4
	+ 0.1110		
逻辑右移	1.0110	110	
	+ 0.1011		



例6.21 结果

① 乘积的符号位 $x_0 \oplus y_0 = 1 \oplus 0 = 1$

② 数值部分按绝对值相乘

$$x^* \cdot y^* = 0.10110110$$

$$\text{则 } [x \cdot y]_{\text{原}} = 1.10110110$$

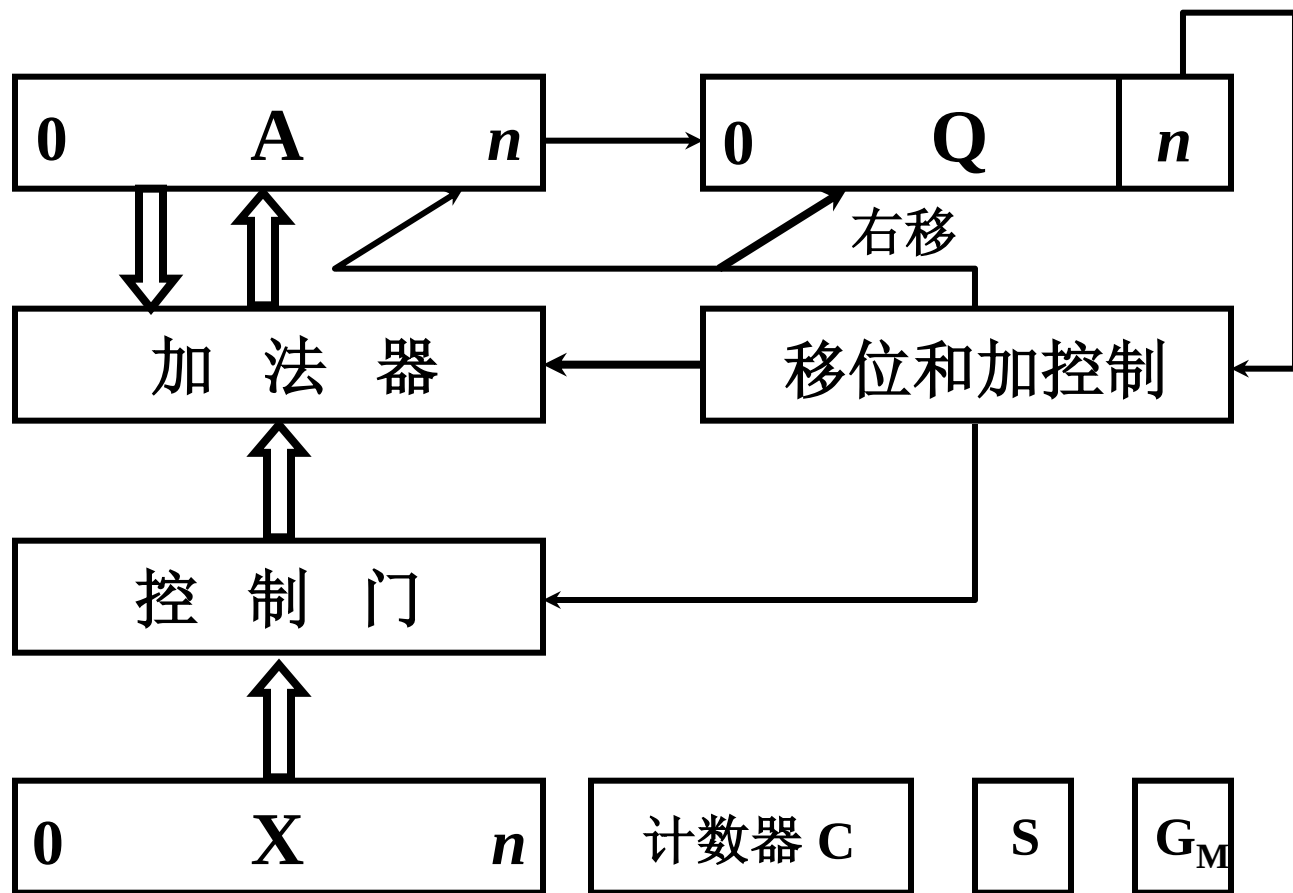
特点 绝对值运算

用移位的次数判断乘法是否结束

逻辑移位



(3) 原码一位乘的硬件配置



A、X、Q 均 $n+1$ 位

移位和加受末位乘数控制



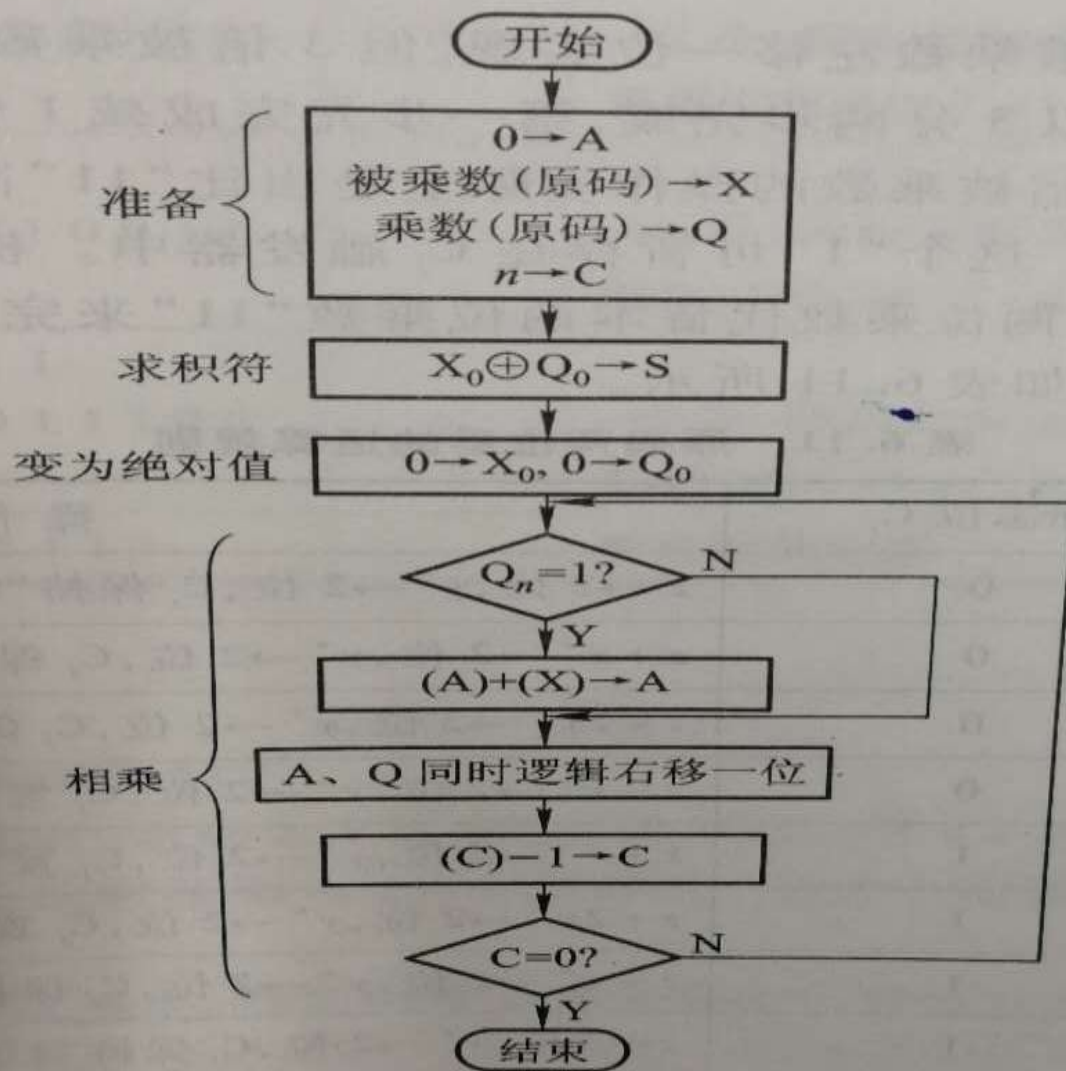


图 6.8 原码一位乘控制流程

(4) 原码两位乘

原码乘

符号位 和 数值位 部分 分开运算

两位乘

每次用 乘数的 2 位判断 原部分积
是否加 和 如何加 被乘数

乘数 $y_{n-1} y_n$	新的部分积
0 0	加 “0” → 2
0 1	加 1 倍的被乘数 → 2
1 0	加 2 倍的被乘数 → 2
1 1	加 3 倍的被乘数 → 2

$$\begin{array}{r} 3 ? \quad \quad 4 \quad 100 \\ \quad \quad \quad \underline{-1} \quad \quad \underline{-01} \\ \quad \quad \quad 3 \quad \quad 11 \end{array}$$

先 减 1 倍 的被乘数
再 加 4 倍 的被乘数



- 为更新部分积，需要这些加减操作： $+x^*$ 、 $-x^*$ 、 $+2x^*$ 、 $+4x^*$
- 其中2倍被乘数可以通过将被乘数左移一位获得；
- 加4倍被乘数的操作，由当前两位11左面的两位乘数于下一轮迭代时代替完成，当前只需要把1暂存在标志触发器中，对触发器置1意味着对高两位乘数加1，即要求高两位乘数替本两位乘数完成加4倍被乘数的操作（下一轮加被乘数和当前加4倍被乘数等价且运算被简化）；

(5) 原码两位乘运算规则

乘数判断位 $y_{n-1}y_n$	标志位 C_j	操作内容
0 0	0	$z \rightarrow 2, y^* \rightarrow 2, C_j$ 保持 “0”
0 1	0	$z+x^* \rightarrow 2, y^* \rightarrow 2, C_j$ 保持 “0”
1 0	0	$z+2x^* \rightarrow 2, y^* \rightarrow 2, C_j$ 保持 “0”
1 1	0	$z-x^* \rightarrow 2, y^* \rightarrow 2, C_j$ 置 “1”
0 0	1	$z+x^* \rightarrow 2, y^* \rightarrow 2, C_j$ 置 “0”
0 1	1	$z+2x^* \rightarrow 2, y^* \rightarrow 2, C_j$ 置 “0”
1 0	1	$z-x^* \rightarrow 2, y^* \rightarrow 2, C_j$ 保持 “1”
1 1	1	$z \rightarrow 2, y^* \rightarrow 2, C_j$ 保持 “1”

共有操作 $+x^*$ $+2x^*$ $-x^*$ $\rightarrow 2$
 实际操作 $+ [x^*]_{\text{补}}$ $+ [2x^*]_{\text{补}}$ $+ [-x^*]_{\text{补}}$ $\rightarrow 2$ 补码移



- 由于减法要用补码操作，参与原码两位乘数值部分计算的两个操作数都要采用绝对值的补码；
- 部分积右移两位的操作按照补码移位规则；
- 计算过程中可能遇到加两倍的被乘数，部分积的绝对值可能大于2，所以对部分积取3位符号位（最高位是真正的符号位）；
- 为统一计算步骤，若乘数是偶数位，需要在乘数前加两个零，如乘数最高的两个有效位出现11时，将 c_i 置1，与填补的两个0形成001状态，完成 $+x^*$ 操作（此步不需要移位）

例6.22 已知 $x = 0.111111$ $y = -0.111001$ 求 $[x \cdot y]_{\text{原}}$

解: 数值部分的运算	乘 数	C_j	说 明
补码右移 000.000000 $+ 000.111111$ 000.111111 000.001111	00.111001 $11\ 001110$	0 0	初态 $z_0 = 0$ $+ x^*,\ C_j = 0$ $\rightarrow 2$
补码右移 $+ 001.111110$ 010.001101 000.100011 $+ 111.000001$	11 $0111\ 0011$	0 0	$+ 2x^*,\ C_j = 0$ $\rightarrow 2$ $- x^*,\ C_j = 1$
补码右移 111.100100 111.111001 $+ 000.111111$ 000.111000	0111 $000111\ 00$ 000111	1 1	$\rightarrow 2$ $+ x^*,\ C_j = 0$



例6.22 结果

① 乘积的符号位 $x_0 \oplus y_0 = 0 \oplus 1 = 1$

② 数值部分的运算

$$x^* \cdot y^* = 0.111000000111$$

$$\text{则 } [x \cdot y]_{\text{原}} = 1.111000000111$$

特点 绝对值的补码运算

用移位的次数判断乘法是否结束

算术移位



(6) 原码两位乘和原码一位乘比较

	原码一位乘	原码两位乘
符号位	$x_0 \oplus y_0$	$x_0 \oplus y_0$
操作数	绝对值	绝对值的补码
移位	逻辑右移	算术右移
移位次数	n	$\frac{n}{2}$ (n 为偶数)
最多加法次数	n	$\frac{n}{2} + 1$ (n 为偶数)

n 为奇数时，在乘数最高位只增加一个0，最后一步右移一位



乘法小结

- 整数乘法与小数乘法完全相同
可用 逗号 代替小数点
- 原码乘 符号位 单独处理
补码乘 符号位 自然形成
- 原码乘去掉符号位运算 即为无符号数乘法
- 不同的乘法运算需有不同的硬件支持



作业 6.5

- 用原码一位乘、两位乘计算：
- $x=0.11011$ $y=-0.10101$
- $x=6$ $y=9$

四、除法运算

1. 分析笔算除法

$$x = -0.1011 \quad y = 0.1101 \quad \text{求 } x \div y$$

$$\begin{array}{r} 0.1101 \overline{) 0.1101} \\ \underline{0.1101} \\ 0.0000 \\ \underline{0.0000} \\ 0.0000 \\ \underline{0.0000} \\ 0.0000 \\ \underline{0.0000} \\ 0.0000 \end{array}$$

✓ 商符单独处理

? 心算上商

? 余数不动低位补“0”
减右移的除数

? 上商位置不固定

$$x \div y = -0.1101 \quad \text{商符心算求得}$$

$$\text{余数 } 0.00000111$$



2. 笔算除法和机器除法的比较

笔算除法

商符单独处理

心算上商：比较余数
(首次是被除数) 和除数
(首次之后是右移后的除数)
的大小确定上为1或者0

余数 不动 低位补 “0”
减右移 的除数

2 倍字长加法器

上商位置 不固定

机器除法

符号位异或形成

求差值： $|x| - |y| > 0$ 上商 1

$|x| - |y| < 0$ 上商 0

余数 左移一位 低位补 “0”
减 除数

1 倍字长加法器

在寄存器 最末位上商，
原有商左移



6. 原码除法

以小数为例

$$[x]_{\text{原}} = x_0.x_1x_2 \dots x_n$$

$$[y]_{\text{原}} = y_0.y_1y_2 \dots y_n$$

$$\left[\frac{x}{y}\right]_{\text{原}} = (x_0 \oplus y_0). \frac{x^*}{y^*}$$

式中 $x^* = 0.x_1x_2 \dots x_n$ 为 x 的绝对值
 $y^* = 0.y_1y_2 \dots y_n$ 为 y 的绝对值

商的符号位单独处理 $x_0 \oplus y_0$

数值部分为绝对值相除 $\frac{x^*}{y^*}$

约定 小数定点除法 $x^* < y^*$ 整数定点除法 $x^* > y^*$

约定被除数不等于 0，除数不能为 0

商的位数运算到和操作数相同



(1) 恢复余数法

- 原码除法按照对余数的处理不同，可分为恢复余数法和不恢复余数（加减交替）法
- 商值的确定是通过比较被除数（余数）和除数的绝对值大小，即求差值
- 恢复余数法的特点：余数（差值）为负时（不够减，应上商0），加上除数，恢复为原来的余数
- 计算机内只设加法器，需要把 $x^* - y^*$ 操作转变为 $[x^*]_{\text{补}} + [-y^*]_{\text{补}}$

例6.24 $x = -0.1011$ $y = -0.1101$ 求 $[\frac{x}{y}]_{\text{原}}$

解: $[x]_{\text{原}} = 1.1011$ $[y]_{\text{原}} = 1.1101$ $[x^*]_{\text{补}} = 0.1011$ $[y^*]_{\text{补}} = 0.1101$

① $x_0 \oplus y_0 = 1 \oplus 1 = 0$ $[-y^*]_{\text{补}} = 1.0011$

② 被除数 (余数)	商	说 明
0.1011 $+ 1.0011$	0.0000	开始: 被除数减除数求余数 $+ [-y^*]_{\text{补}}$
1.1110 $+ 0.1101$	0	余数为负, 上商 0 恢复余数 $+ [y^*]_{\text{补}}$
0.1011 1.0110 $+ 1.0011$	0 0	恢复后的余数 $\leftarrow 1$ $+ [-y^*]_{\text{补}}$
0.1001 1.0010 $+ 1.0011$	01 01	余数为正, 上商 1 $\leftarrow 1$ $+ [-y^*]_{\text{补}}$

逻辑左移

逻辑左移



被除数（余数）	商	说 明
0 . 0 1 0 1	0 1 1	余数为正，上商 1
逻辑左移 0 . 1 0 1 0	0 1 1	← 1
+ 1 . 0 0 1 1		+ $[-y^*]_{\text{补}}$
1 . 1 1 0 1	0 1 1 0	余数为负，上商 0
+ 0 . 1 1 0 1		恢复余数 + $[y^*]_{\text{补}}$
0 . 1 0 1 0	0 1 1 0	恢复后的余数
逻辑左移 1 . 0 1 0 0	0 1 1 0	← 1
+ 1 . 0 0 1 1		+ $[-y^*]_{\text{补}}$
0 . 0 1 1 1	0 1 1 0 1	余数为正，上商 1

$$\frac{x^*}{y^*} = 0.1101 \quad \text{上商 5 次} \quad \text{左移 4 次}$$

$$\therefore \left[\frac{x}{y} \right]_{\text{原}} = 0.1101 \quad \text{第一次上商在整数位，判溢出}$$

余数为正，上商 1；左移余数，减除数计算新余数

余数为负，上商 0；加除数以恢复余数；左移余数，减除数计算新余数，恢复余数增加用时



(2) 不恢复余数法（加减交替法）

- 恢复余数法运算规则

余数 $R_i > 0$ 上商 “1”， $2R_i - y^*$

余数 $R_i < 0$ 上商 “0”， $R_i + y^*$ 恢复余数

$$2(R_i + y^*) - y^* = 2R_i + y^*$$

- 不恢复余数法运算规则

$R_i > 0$: 上商 “1” $2R_i - y^*$

$R_i < 0$: 上商 “0” $2R_i + y^*$

首次计算余数是减法，
以后根据上商值加减交替



例6.25 $x = -0.1011$ $y = -0.1101$ 求 $[\frac{x}{y}]_{\text{原}}$

解:	0.1011	0.0000	
	+1.0011		$+[-y^*]_{\text{补}}$
	1.1110	0	余数为负, 上商 0
逻辑左移	1.1100	0	$\leftarrow 1$
逻辑左移	+0.1101		$+ [y^*]_{\text{补}}$
	0.1001	01	余数为正, 上商 1
逻辑左移	1.0010	01	$\leftarrow 1$
逻辑左移	+1.0011		$+ [-y^*]_{\text{补}}$
	0.0101	011	余数为正, 上商 1
逻辑左移	0.1010	011	$\leftarrow 1$
逻辑左移	+1.0011		$+ [-y^*]_{\text{补}}$
	1.1101	0110	余数为负, 上商 0
逻辑左移	1.1010	0110	$\leftarrow 1$
逻辑左移	+0.1101		$+ [y^*]_{\text{补}}$
	0.0111	01101	余数为正, 上商 1

$$[x]_{\text{原}} = 1.1011$$

$$[y]_{\text{原}} = 1.1101$$

$$[x^*]_{\text{补}} = 0.1011$$

$$[y^*]_{\text{补}} = 0.1101$$

$$[-y^*]_{\text{补}} = 1.0011$$



例6.25 结果

$$\textcircled{1} x_0 \oplus y_0 = 1 \oplus 1 = 0$$

$$\textcircled{2} \frac{x^*}{y^*} = 0.1101$$

$$\therefore \left[\frac{x}{y} \right]_{\text{原}} = 0.1101$$

特点 加（减） $n+1$ 次，上商 $n+1$ 次，

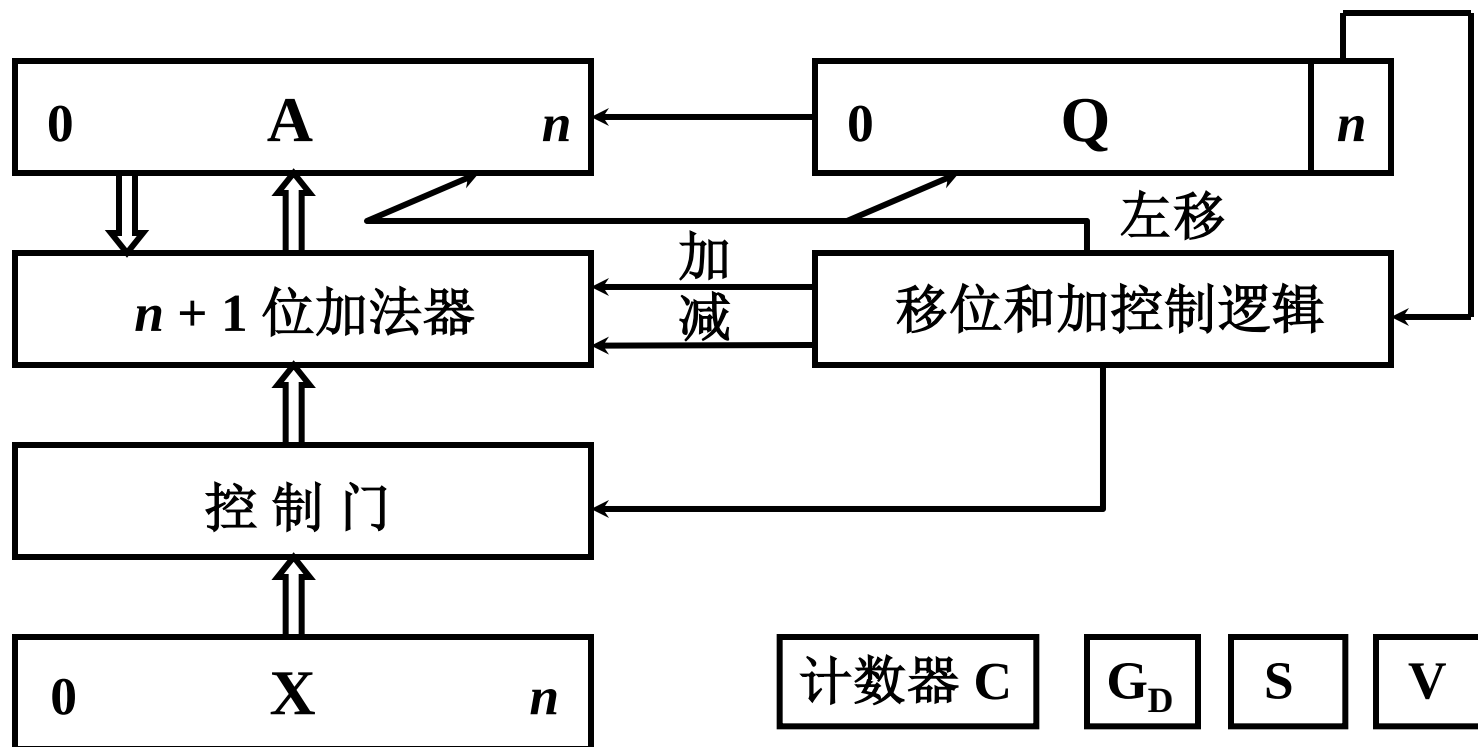
第一次上商判溢出

余数、商左移 n 次

用移位的次数判断除法是否结束



(3) 原码加减交替除法硬件配置



A、X、Q 均 $n+1$ 位

用本次上商值 Q_n 控制加减交替



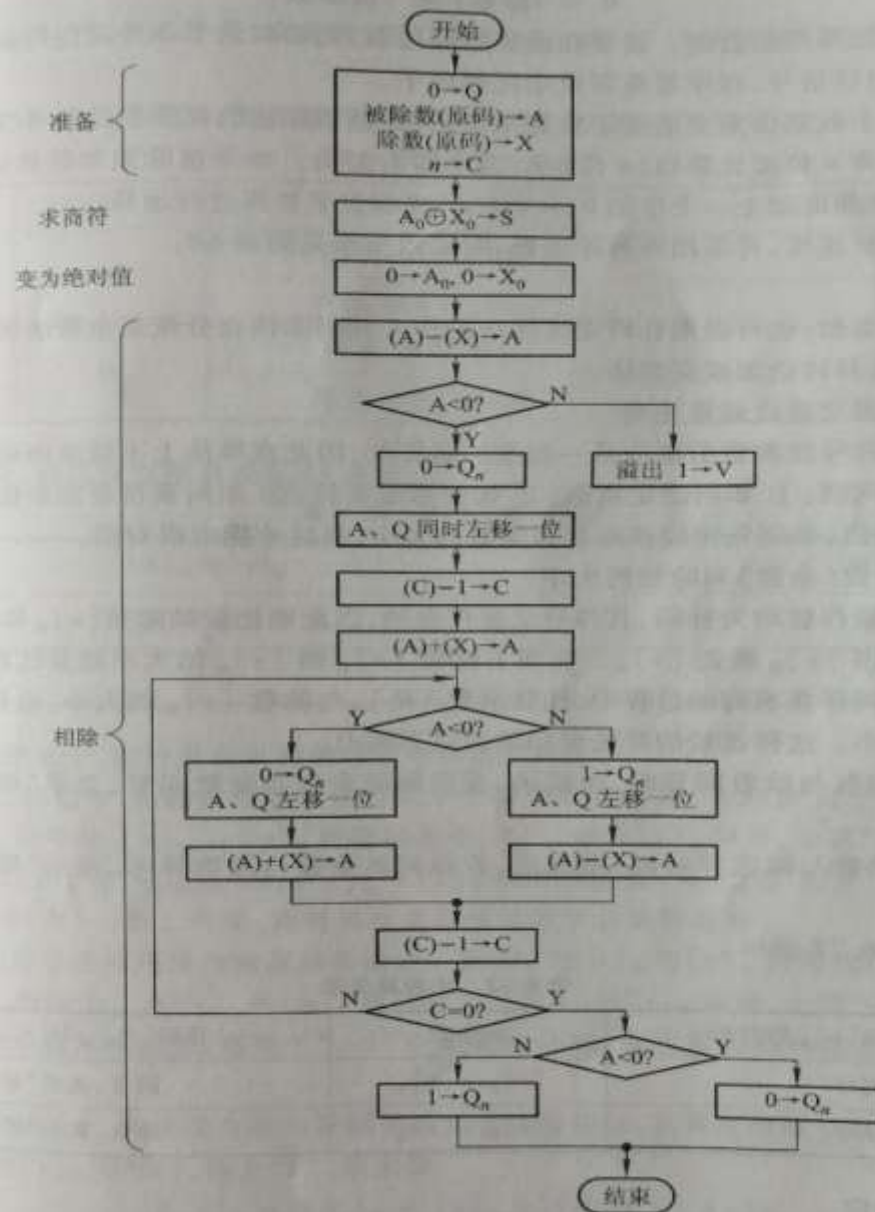


图 6.12 原码加减交替法控制流程图

作业 6.6

- 用原码交替加减法计算，被除数和除数分别是：

$$x = \frac{13}{32}, y = -\frac{27}{32}$$

6.4 浮点四则运算

一、浮点加减运算 $x = S_x \cdot 2^{j_x} \quad y = S_y \cdot 2^{j_y}$

- 浮点数包括尾数和阶码两部分，只有阶码相等时，才能把尾数直接相加减；
- 浮点数加减包含如下步骤：对阶、尾数求和、规格化、舍入、溢出判断。
- 本部分内容假设浮点数尾数及阶码都采用补码表示。



浮点加减运算步骤:

$$x = S_x \cdot 2^{j_x} \quad y = S_y \cdot 2^{j_y}$$

1. 对阶

(1) 求阶差

$$\Delta j = j_x - j_y = \begin{cases} = 0 & j_x = j_y & \text{已对齐} \\ > 0 & j_x > j_y & \begin{cases} x \text{ 向 } y \text{ 看齐} & S_x \leftarrow 1, j_x - 1 \\ y \text{ 向 } x \text{ 看齐} & \checkmark S_y \rightarrow 1, j_y + 1 \end{cases} \\ < 0 & j_x < j_y & \begin{cases} x \text{ 向 } y \text{ 看齐} & \checkmark S_x \rightarrow 1, j_x + 1 \\ y \text{ 向 } x \text{ 看齐} & S_y \leftarrow 1, j_y - 1 \end{cases} \end{cases}$$

(2) 对阶原则

小阶向大阶看齐, 保证尾数是纯小数, 小阶浮点数的尾数右移次数等于阶差



例如 $x = 0.1101 \times 2^{01}$ $y = (-0.1010) \times 2^{11}$

求 $x+y$

解: $[x]_{\text{补}} = 00, 01; 00.1101$ $[y]_{\text{补}} = 00, 11; 11.0110$

1. 对阶

① 求阶差 $[\Delta j]_{\text{补}} = [j_x]_{\text{补}} - [j_y]_{\text{补}} = 00, 01$

$$\begin{array}{r} + \quad 11, 01 \\ \hline 11, 10 \end{array}$$

阶差为负 (-2) $\therefore S_x \rightarrow 2 \quad j_x + 2$

② 对阶 $[x]_{\text{补}}' = 00, 11; 00.0011$

2. 尾数求和

$$\begin{array}{r} [S_x]_{\text{补}}' = 00.0011 \quad \text{对阶后的}[S_x]_{\text{补}}' \\ + \quad [S_y]_{\text{补}} = 11.0110 \\ \hline 11.1001 \\ \therefore [x+y]_{\text{补}} = 00, 11; 11. 1001 \end{array}$$



3. 规格化

(1) 规格化数的定义

$$r = 2 \quad \frac{1}{2} \leq |S| < 1$$

尾数绝对值在0.5和1之间
即小数点后第一位是1

(2) 规格化数的判断

$S > 0$ 规格化形式

真值 $0.1 \times \times \dots \times$

原码 $0.\boxed{1} \times \times \dots \times$

补码 $\boxed{0.1} \times \times \dots \times$

反码 $0.1 \times \times \dots \times$

$S < 0$ 规格化形式

真值 $-0.1 \times \times \dots \times$

原码 $1.\boxed{1} \times \times \dots \times$

补码 $\boxed{1.0} \times \times \dots \times$

反码 $1.0 \times \times \dots \times$

原码 不论正数、负数，第一数位为1

补码 符号位和第一数位不同



为便于硬件判断，对 $S < 0$ 的情况规定了两个特例：

$$S = -\frac{1}{2} = -0.100 \cdots 0$$

$$[S]_{\text{原}} = \boxed{1.1} 0 0 \cdots 0$$

$$[S]_{\text{补}} = 1.1 0 0 \cdots 0$$

$\therefore [-\frac{1}{2}]_{\text{补}}$ 不是规格化的数

$$S = -1$$

$$[S]_{\text{补}} = \boxed{1.0} 0 0 \cdots 0$$

$\therefore [-1]_{\text{补}}$ 是规格化的数



规格化操作规则:

- ✓ 如前所述，当尾数(补码)求和出现：
00.0xxxx或者11.1xxxx时，意味着：尾数是纯小数，但是小数点后第一位是0，需要左规格化
- ✓ 另外，当尾数(补码)求和出现：01.xxxxx或者10.xxxxx时，这实质上是两个尾数的和超过1，产生溢出，意味着：尾数不是纯小数，小数点左面整数位是1，需要向右规格化（对于定点小数，就是溢出）

(3) 左规（对应尾数第一位真值不是1情况）

尾数每左移1位，阶码减 1，直到数符和第一数位不同为止

上例 $[x+y]_{\text{补}} = 00, 11; 11. 1001$

左规后 $[x+y]_{\text{补}} = 00, 10; 11. 0010$

$$\therefore x + y = (-0.1110) \times 2^{10}$$

(4) 右规（对应尾数绝对值大于1的情况）

当 尾数溢出（>1）时，需 右规

即尾数出现 $01. \times \times \cdots \times$ 或 $10. \times \times \cdots \times$ 时

尾数右移一位，阶码加 1



例6.27 $x = 0.1101 \times 2^{10}$ $y = 0.1011 \times 2^{01}$

求 $x+y$ (除阶符、数符外, 阶码取 3 位, 尾数取 6 位)

解: $[x]_{\text{补}} = 00, 010; 00. 110100$
 $[y]_{\text{补}} = 00, 001; 00. 101100$

① 对阶

$$[\Delta j]_{\text{补}} = [j_x]_{\text{补}} - [j_y]_{\text{补}} = \begin{array}{r} 00, 010 \\ + 11, 111 \\ \hline 100, 001 \end{array}$$

阶差为 +1 $\therefore S_y \rightarrow 1, j_y + 1$

$$\therefore [y]_{\text{补}}' = 00, 010; 00. 010110$$

② 尾数求和

$$\begin{array}{r} [S_x]_{\text{补}} = 00. 110100 \\ + [S_y]_{\text{补}}' = 00. 010110 \\ \hline 01. 001010 \end{array} \quad \begin{array}{l} \text{对阶后的 } [S_y]_{\text{补}}' \\ \text{尾数溢出需右规} \end{array}$$



③ 右规

$$[x+y]_{\text{补}} = 00, 010; 01. 001010$$

右规后

$$[x+y]_{\text{补}} = 00, 011; 00. 100101$$

$$\therefore x+y = 0. 100101 \times 2^{11}$$

4. 舍入

在 对阶 和 右规 过程中，尾数右移可能出现 尾数末位丢失 引起误差，需考虑舍入

(1) 0 舍 1 入法

(2) 恒置 “1” 法



例 6.28 $x = (-\frac{5}{8}) \times 2^{-5}$ $y = (-\frac{7}{8}) \times 2^{-4}$

求 $x-y$ (除阶符、数符外, 阶码取 3 位, 尾数取 6 位)

解: $x = (-0.101000) \times 2^{-101}$ $y = (0.111000) \times 2^{-100}$

$[x]_{\text{补}} = 11, 011; 11. 011000$ $[y]_{\text{补}} = 11, 100; 00. 111000$

① 对阶

$$\begin{array}{rcl} [\Delta j]_{\text{补}} = [j_x]_{\text{补}} - [j_y]_{\text{补}} & = & 11, 011 \\ & + & 00, 100 \\ \hline & & 11, 111 \end{array}$$

阶差为 -1 $\therefore S_x \xrightarrow{-1} j_x + 1$

$\therefore [x]_{\text{补}}' = 11, 100; 11. 101100$



② 尾数求和

$$\begin{array}{r} [S_x]_{\text{补}'} = 11.101100 \\ + [-S_y]_{\text{补}} = 11.001000 \\ \hline 110.110100 \end{array}$$

③ 右规

$$[x - y]_{\text{补}} = 11, 100; 10.110100$$

右规后

$$[x - y]_{\text{补}} = 11, 101; 11.011010$$

$$\therefore x - y = (-0.100110) \times 2^{-11}$$

$$= \left(-\frac{19}{32}\right) \times 2^{-3}$$



5. 溢出判断

- 浮点运算最后一步需要判断溢出，对规格化数的阶码进行判断
- 若补码阶码数位为 n 位，则表示范围是 -2^n 至 $+2^n - 1$ ，运算结果的阶码超出此范围导致上溢出和下溢出
- 浮点数溢出与否的判断：
 - 阶码补码=01,xxx...是上溢出，做中断溢出处理
 - 阶码补码=10,xxx...是下溢出，按机器零处理
- 尾数出现：01.xxx...或者10.xxx...不表示溢出，是需要做右规，尾数右规改变阶码可能导致溢出

Review 浮点数加减运算步骤

1. 对阶 小阶向大阶看齐

2. 尾数求和 采用双符号位补码运算

6. 规格化 使得 $1/2 \leq |\text{求和之后的尾数}| < 1$

✓ 规格化尾数的判断(补码): 符号位和最高数值位不能一致; 两位符号位应该一致

✓ 规格化操作: 当 $|\text{尾数}| < 1/2$ 时, 出现数符和最高数值位相同, 需左规, 尾数左移1位, 阶码减1, 直到数符和最高数值位不同为止; 当 $|\text{尾数}| > 1$, 出现 **01.** × × × 或 **10.** × × × 时, 需右规, 尾数右移1位, 阶码加1

4. 舍入

5. 溢出判断

二、浮点乘除运算

$$x = S_x \cdot 2^{j_x} \quad y = S_y \cdot 2^{j_y}$$

1. 乘法

$$x \cdot y = (S_x \cdot S_y) \times 2^{j_x + j_y}$$

2. 除法

$$\frac{x}{y} = \frac{S_x}{S_y} \times 2^{j_x - j_y}$$

6. 步骤

- (1) 对阶码使用定点加（乘法）减（除法）运算
- (2) 对尾数使用定点乘除运算
- (3) 需要进行规格化



4. 浮点运算部件

- 浮点四则运算，阶码只有加减运算；尾数有加减乘除运算
- 浮点运算器由两个定点运算部件组成：阶码运算部件，尾数运算部件
- 计算机中浮点运算部件可以做成独立的选件，即浮点协处理器

作业 6.7

- 按照补码浮点运算规则计算，除阶符、数符外，阶码取 3 位，尾数取 6 位：

$$1. \quad \left[2^{-3} \times \frac{13}{16} \right] - \left[2^{-4} \times \left(-\frac{5}{8} \right) \right]$$

$$2. \quad \left[2^5 \times \frac{5}{8} \right] + \left[2^3 \times \left(\frac{9}{16} \right) \right]$$

Thank you